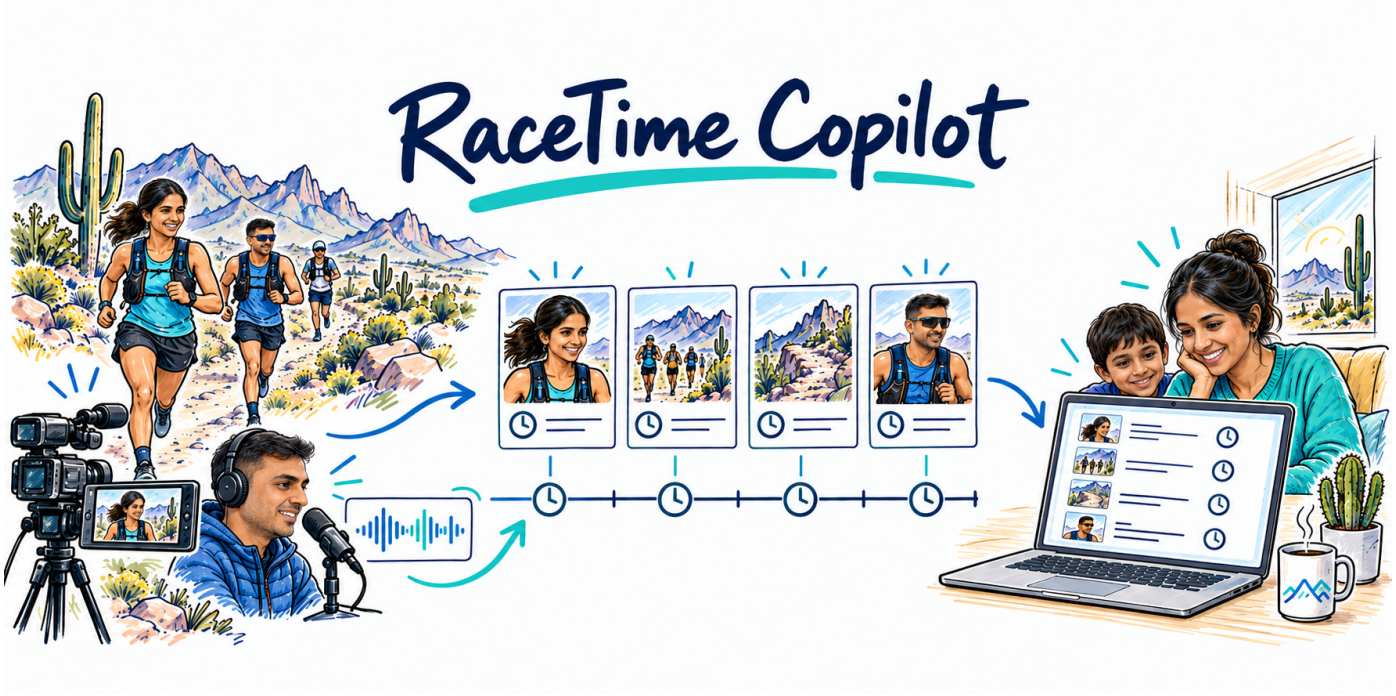
1 / THE PROBLEM

Missed 15 minutes? Catch up.

RaceTime Copilot helps you understand a chosen race interval through timestamped evidence.



The problem

Long broadcasts are hard to catch up on. Scrubbing can miss key moments or reveal future results.

The useful result

Choose a window, follow a runner, and see what the sources reported - including disagreements.

What works today

Imported captions and observations. Gemini is deferred; a video URL alone is not analyzed.

2 / WHO IT HELPS

For the people following the race.

Built by Siva Babu: ultramarathoner, race organizer and observability practitioner.

Fans and families

Catch up together on a missed interval. Inspect the evidence behind a reported sighting or position.

Organizers and crews

Review imported commentary and timing context. Keep uncertain reports visible.

Why this project

A personal race-viewing problem meets systems skills: time boundaries, memory limits and useful traces.

3 / HOW IT WORKS

One question. One evidence workflow.

Import VTT, SRT or JSON observations. Choose an interval and spoiler cutoff, then ask what happened.

Retrieve	Keep only evidence fully inside the window.	Rank relevant observations; retry one transient retrieval failure.
Check	Show annotated claims that disagree.	Exclude later evidence. Say when there is not enough evidence.
Review	Read the timestamped recap and its trace.	Approve or reject the saved result; export JSON when useful.

Current mode: rule-driven LangGraph + extractive text. No live LLM planner or automatic video watching.

4 / TECHNOLOGY AND LEARNING

Each tool has a clear job.

Five course themes applied to one product. The detailed technology map is in docs/technology-map.md.

Week 1 / App	Streamlit + Python requests; TypeScript + Zod.	A local screen calls one validated API. React is an optional second interface.
Week 2 / Retrieval	VTT/SRT/JSON, lexical ranking and hashed vectors.	Find time-bounded evidence. Learned embeddings and LLM synthesis remain future work.
Week 3 / Workflow	LangGraph, D1/SQLite and byte-weighted LRU.	Trace steps, retry, store review and reuse stable requests. No autonomous planning or durable graph pause yet.
Week 4 / Evaluation	Node tests, Streamlit AppTest and local traces.	Measure behavior against synthetic cases. Independent labels and external LangSmith verification remain.
Week 5 / LoRA	PyTorch, Transformers, PEFT and scikit-learn.	Train, evaluate and merge a BERT-tiny adapter. This adapts the technique, not the exact Qwen3/LLaMA Factory setup.

Gemini is the planned video-to-evidence adapter. WebMCP is available in the optional browser interface.

5 / EVIDENCE THAT IT WORKS

Small tests. Clear limits.

These measurements use authored synthetic data. They do not establish real-video summary quality.

17 / 17	40 / 40	Passed
Unit tests	Evidence cases	Streamlit + API flows

Spoiler test	Request 00:00-15:00 of fictional Canyon Relay.	Two ridge reports disagree. The update at 15:30 stays out.
Evidence result	Naive baseline 20/40 -> workflow 40/40.	Strict time and availability rules explain the measured improvement.
Separate LoRA lab	52.5% baseline -> 70% held-out accuracy.	144 training / 40 held-out examples. The app keeps the rule router; the trained adapter is not deployed.

Also checked: import, live append, revision conflicts, session isolation, review, retry, cache and history.

Reproduce: npm test | npm run eval | python tests/streamlit_smoke.py (with the demo running). Reports are in reports/.

6 / RUN IT AND EXTEND IT

A simple demo. A clear next step.

Requires Python 3.11+ and Node.js 22.13+. No API key is needed for the supported evidence workflow.

START LOCALLY

```
python3 -m venv .venv
source .venv/bin/activate
pip install -r requirements.txt
npm ci
python scripts/run_demo.py
```

Open http://localhost:8501

Try this first

Choose Canyon Relay. Ask What happened? for 00:00-15:00 with cutoff 15:00. Inspect the conflicting reports, approve or reject, then open the trace.

Canyon Relay is fictional. Live mode currently means appending evidence, not watching a livestream automatically.

Next	Connect bounded Gemini video observations.	Check timestamps and claims against independently reviewed video intervals.
Then	Pilot with race fans and organizers.	Measure time saved and evidence accuracy. Explore timing feeds, runner watchlists and multi-camera recaps.

github.com/sivalinb/racetime-copilot

Read: README -> docs/learning-guide.md -> docs/demo-guide.md. Course scope: user-provided Week 1-5 handouts.